

Test Case 1: Leader Election

Goal

- Demonstrate that when your Python Raft nodes (1-5) first start up, **exactly one** node wins the leader election and the others become followers.

Procedure

1. **Start** the Python cluster (e.g., `cd Q3_python && docker compose up`).
2. **Observe** the console logs for nodes 1-5.
3. Within a few seconds, one node should print something like:

```
[Node 3] Became CANDIDATE in term 1, ...  
[Node 3] Won election with 3 votes: becoming LEADER for term 1.
```

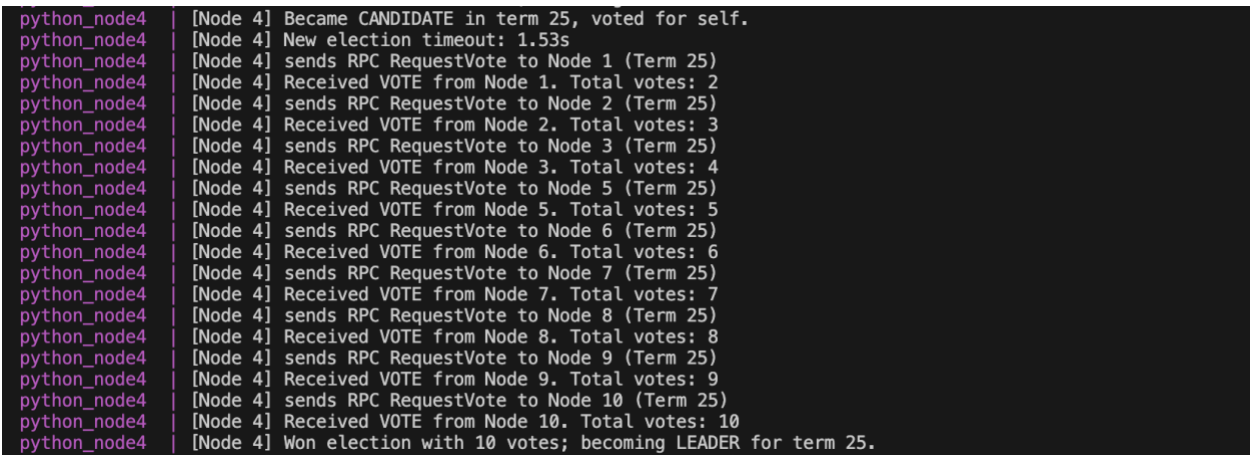
(The actual node number might vary.)

4. The other Python nodes should then say something like “Not enough votes; reverting to FOLLOWER” or “Heartbeat accepted.”

Expected Outcome

- One Python node prints “Won election... becoming LEADER.”
- No other Python node claims leadership in that term.

Screenshot



```
python_node4 | [Node 4] Became CANDIDATE in term 25, voted for self.  
python_node4 | [Node 4] New election timeout: 1.53s  
python_node4 | [Node 4] sends RPC RequestVote to Node 1 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 1. Total votes: 2  
python_node4 | [Node 4] sends RPC RequestVote to Node 2 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 2. Total votes: 3  
python_node4 | [Node 4] sends RPC RequestVote to Node 3 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 3. Total votes: 4  
python_node4 | [Node 4] sends RPC RequestVote to Node 5 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 5. Total votes: 5  
python_node4 | [Node 4] sends RPC RequestVote to Node 6 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 6. Total votes: 6  
python_node4 | [Node 4] sends RPC RequestVote to Node 7 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 7. Total votes: 7  
python_node4 | [Node 4] sends RPC RequestVote to Node 8 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 8. Total votes: 8  
python_node4 | [Node 4] sends RPC RequestVote to Node 9 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 9. Total votes: 9  
python_node4 | [Node 4] sends RPC RequestVote to Node 10 (Term 25)  
python_node4 | [Node 4] Received VOTE from Node 10. Total votes: 10  
python_node4 | [Node 4] Won election with 10 votes; becoming LEADER for term 25.
```

Test Case 2: Re-Election if Elected Leader Fails

Goal

- Show that if the Python leader crashes or is stopped, the cluster detects the missing heartbeats and elects a **new** leader automatically.

Procedure

1. **Identify** the current leader from Test Case 1 (say node3).
2. **Stop** that container. For example:

```
docker stop python node3
```

3. **Observe** the remaining Python nodes' logs for an election timeout (no heartbeats from the old leader).
4. After the timeout, one of the followers becomes a candidate and holds a new election. Eventually, it should declare:

```
[Node 2] Won election with 3 votes; becoming LEADER for term 2
```

- Optionally, **restart** the old leader node (e.g., `docker start python_node3`), and watch it rejoin as a follower.

Expected Outcome

- A **new** leader emerges. The console logs show:
 - “No heartbeat for Xs; starting election.”
 - Then “Became CANDIDATE in term 2” and “Won election...”.
- The old leader, once restarted, does **not** override the new leader; it stays follower.

Screenshot

```
python_node3 exited with code 137
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 runs RPC AppendEntries called by Node 4 (Term 25)
python_node3 Node 31 Heartbeat/log accepted from leader 4 (Term 25)
python_node3 Node 31 No heartbeat for 2.34s: starting election
python_node3 Node 31 Became CANDIDATE in term 26, voted for self.
python_node3 Node 31 New election timeout: 1.67s
python_node3 Node 31 sends RPC RequestVote to Node 1, Total votes: 1
python_node3 Node 31 Received VOTE from Node 1, Total votes: 2
python_node3 Node 31 sends RPC RequestVote to Node 2, Total votes: 3
python_node3 Node 31 Received VOTE from Node 2, Total votes: 3
python_node3 Node 31 sends RPC RequestVote to Node 4 (Term 26)
python_node3 Node 31 failed to send RequestVote to Node 4, Error: <Inactive>cdcr of RPC that terminated with:
python_node3 status = STATUS_CODE.UNAVAILABLE
python_node3 details = "DNS resolution failed for node:58054: C-ares status is notARES_SUCCESS qtype=a name=node4 is_balanced=0: Do
python_node3 main name not found"
python_node3 debug_error_string = "UNKNOWN:DNS resolution failed for node:58054: C-ares status is notARES_SUCCESS qtype=a name=node
python_node3 4 is_balanced=0: Domain name not found (UNKNOWN:DNS-46-147211:87:13:573633514+00:00, grpc-status=14)"
python_node3 =
python_node3 Node 31 sends RPC RequestVote to Node 5 (Term 26)
python_node3 Node 31 Received VOTE from Node 5, Total votes: 4
python_node3 Node 31 sends RPC RequestVote to Node 6 (Term 26)
python_node3 Node 31 Received VOTE from Node 6, Total votes: 5
python_node3 Node 31 sends RPC RequestVote to Node 7 (Term 26)
python_node3 Node 31 Received VOTE from Node 7, Total votes: 6
python_node3 Node 31 sends RPC RequestVote to Node 8 (Term 26)
python_node3 Node 31 Received VOTE from Node 8, Total votes: 7
python_node3 Node 31 sends RPC RequestVote to Node 9 (Term 26)
python_node3 Node 31 Received VOTE from Node 9, Total votes: 8
python_node3 Node 31 sends RPC RequestVote to Node 10 (Term 26)
python_node3 Node 31 Received VOTE from Node 10, Total votes: 9
python_node3 Node 31 Won election with 9 votes! Becoming LEADER in term 26.
```

Test Case 3: Log Replication to Go Nodes

Goal

- Verify that once the Python side is the sole leader, it replicates **actual log entries** to the Go nodes.
- The Go logs should show “Log size = X” after receiving these entries.

Procedure

1. **Start** both clusters:
 - `docker compose up` in Q3_python (Python nodes 1..5).
 - `docker compose up` in Q4_golang (Go nodes 6..10).
2. **Wait** until a single Python node becomes leader. Check logs for “Won election...”.
3. **Send** a client request to the Python leader, e.g.:

```
grpcurl -plaintext \
-d '{"operation": "PUT x=5"}' \
localhost:50055 \
raft.Raft/SendClientRequest
```

(Adjust for your actual leader’s port; if node5 is leader, it might be 50055:50055. Or run a Python/Go test script.)

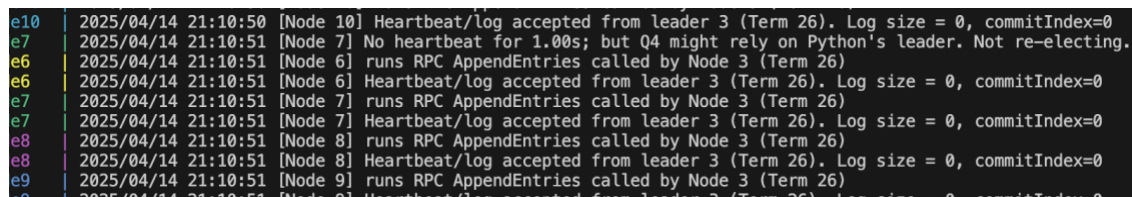
4. **Observe** the Python leader logs
5. **Observe** the Go nodes (6..10) logs for:

```
[Node 6] runs RPC AppendEntries called by Node 5 (Term X)
[Node 6] Heartbeat/log accepted from leader 5. Log size = 1,
commitIndex=1
```

Expected Outcome

- Python leader prints logs indicating it appended the new operation and invoked `AppendEntries` for each node 6..10.
- Each Go node prints “Log size = 1” (or increments if you do multiple requests).

Screenshot



```
e10 | 2025/04/14 21:10:50 [Node 10] Heartbeat/log accepted from leader 3 (Term 26). Log size = 0, commitIndex=0
e7 | 2025/04/14 21:10:51 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
e6 | 2025/04/14 21:10:51 [Node 6] runs RPC AppendEntries called by Node 3 (Term 26)
e6 | 2025/04/14 21:10:51 [Node 6] Heartbeat/log accepted from leader 3 (Term 26). Log size = 0, commitIndex=0
e7 | 2025/04/14 21:10:51 [Node 7] runs RPC AppendEntries called by Node 3 (Term 26)
e7 | 2025/04/14 21:10:51 [Node 7] Heartbeat/log accepted from leader 3 (Term 26). Log size = 0, commitIndex=0
e8 | 2025/04/14 21:10:51 [Node 8] runs RPC AppendEntries called by Node 3 (Term 26)
e8 | 2025/04/14 21:10:51 [Node 8] Heartbeat/log accepted from leader 3 (Term 26). Log size = 0, commitIndex=0
e9 | 2025/04/14 21:10:51 [Node 9] runs RPC AppendEntries called by Node 3 (Term 26)
e9 | 2025/04/14 21:10:51 [Node 9] Heartbeat/log accepted from leader 3 (Term 26). Log size = 0, commitIndex=0
```

Test Case 4: Leader Sends AppendEntries to All Remaining Nodes

Goal

- Demonstrate that the Python leader periodically sends **heartbeat** (AppendEntries) calls to **all** nodes in the cluster (1..10), including the other Python followers and all Go nodes.

Procedure

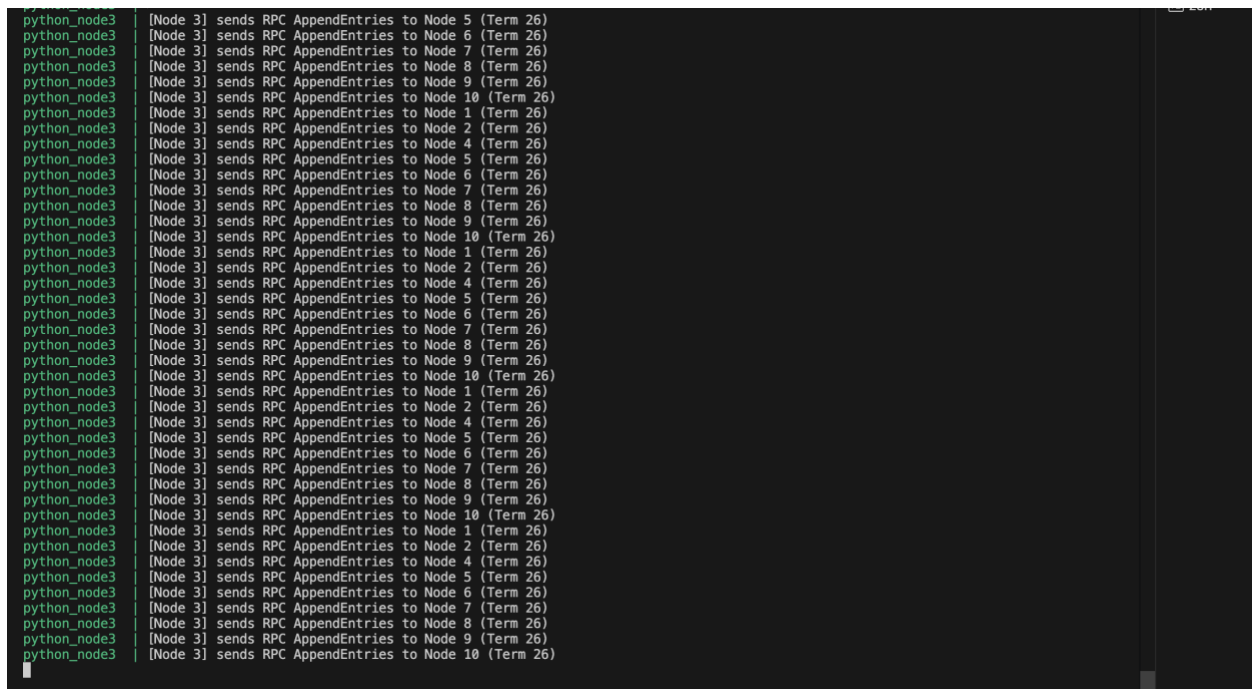
1. With the cluster running and a known Python leader, **watch** the logs of all containers.
2. The leader's `run_heartbeat_timer()` calls `replicate_log_to_all()` (or something similar) **every 1 second** by default.
3. Each time, you should see in the leader's logs:

```
[Node 5] sends RPC AppendEntries to Node 1 (Term X)
[Node 5] sends RPC AppendEntries to Node 2 (Term X)
[Node 5] sends RPC AppendEntries to Node 10 (Term X)
```

Expected Outcome

- Leader prints repeated “sends RPC AppendEntries” lines.
- All other nodes print “runs RPC AppendEntries... Heartbeat/log accepted.”

Screenshot



```
python_node3 [Node 3] sends RPC AppendEntries to Node 5 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 6 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 7 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 8 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 9 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 10 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 1 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 2 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 4 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 5 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 6 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 7 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 8 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 9 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 10 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 1 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 2 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 4 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 5 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 6 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 7 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 8 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 9 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 10 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 1 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 2 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 4 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 5 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 6 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 7 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 8 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 9 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 10 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 1 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 2 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 4 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 5 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 6 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 7 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 8 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 9 (Term 26)
python_node3 [Node 3] sends RPC AppendEntries to Node 10 (Term 26)
```

Test Case 5: Go Lang Uses Python Leader and Does Not Conduct Its Own Election

Goal

- Confirm the Go nodes are **not** fully electing their own leader, instead deferring to the Python-chosen leader.
- This ensures that the Go cluster remains in a follower role, using the Python leader for any client requests.

Procedure

1. **Launch** the Go cluster (6..10) and Python cluster (1..5).
2. Inspect the Go logs for messages like:

```
[Node 6] No heartbeat for 2.00s; but Q4 might rely on Python's leader.
Not re-electing.
```

3. Once Python picks a leader, it sends heartbeats to node6..node10.
4. The Go logs show they receive these heartbeats and set `leaderID` to the Python leader, but do **not** attempt to become candidate or print “Became candidate in term...”.

Expected Outcome

- The Go nodes **do not** hold a majority election among themselves.
- Instead, each time they see no heartbeat, they might do nothing, waiting for Python’s leader.
- As soon as Python leader heartbeats arrive, they accept them and remain followers.

Screenshot

```
go_node7 2025/04/14 21:14:12 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:12 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:12 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:12 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:12 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node10 2025/04/14 21:14:13 [Node 10] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node9 2025/04/14 21:14:13 [Node 9] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node8 2025/04/14 21:14:13 [Node 8] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node6 2025/04/14 21:14:13 [Node 6] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node10 2025/04/14 21:14:13 [Node 10] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node6 2025/04/14 21:14:13 [Node 6] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node9 2025/04/14 21:14:13 [Node 9] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node8 2025/04/14 21:14:13 [Node 8] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node10 2025/04/14 21:14:13 [Node 10] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node6 2025/04/14 21:14:13 [Node 6] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node9 2025/04/14 21:14:13 [Node 9] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node8 2025/04/14 21:14:13 [Node 8] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node6 2025/04/14 21:14:13 [Node 6] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node10 2025/04/14 21:14:13 [Node 10] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:13 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node9 2025/04/14 21:14:13 [Node 9] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node8 2025/04/14 21:14:13 [Node 8] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node6 2025/04/14 21:14:13 [Node 6] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node10 2025/04/14 21:14:13 [Node 10] No heartbeat for 2.00s; but Q4 might rely on Python's leader. Not re-electing.
go_node7 2025/04/14 21:14:14 [Node 7] No heartbeat for 1.00s; but Q4 might rely on Python's leader. Not re-electing.
```

Ln 257, Col 26 Spaces: 4 UTF-8 LF () Python 3.13.0 @ Go Live

